



An integrated fault tolerance technique for combinational circuits based on implications and transistor sizing



Ahmad T. Sheikh^a, Aiman H. El-Maleh^{b,*}

^a College of Computer Science & Engineering, King Fahd University of Petroleum & Minerals (KFUPM), Dhahran 31261, Saudi Arabia

^b Department of Computer Engineering, King Fahd University of Petroleum & Minerals (KFUPM), Dhahran 31261, Saudi Arabia

ARTICLE INFO

Keywords:

Logic implications
Functionally redundant wire
Single event transient (SET)
Single event Upset (SEU)
Soft-error tolerance

ABSTRACT

With fabrication technology reaching nano levels, systems are exposed to higher susceptibility to soft errors. Thus, development of effective techniques for designing soft error tolerant systems is of high importance. In this work, an integrated soft error tolerance technique based on logical implications and transistor sizing is proposed. In order to reduce implication learning time, a set of source and target nodes with predefined thresholds are selected and implications between these nodes are extracted. Then, the impact of adding a functionally redundant wire (FRW) due to each implication is evaluated. This is done based on identifying an implication path and the gates along the implication path whose detection probabilities will be reduced due to adding the implication FRW. Then, the gain of an implication is estimated in terms of reduction in fault detection probabilities of gates along an implication path. The implication with the highest gain is selected. The process is repeated until the gain is less than a predetermined threshold. The proposed implication-based fault tolerance technique enhances the circuit reliability with minimal area overhead based on enhancing logical masking. However, its effectiveness depends on the existence of such relations in a circuit and can enhance circuit reliability upto a certain level. To enhance circuit reliability to any required level, selective-transistor redundancy (STR) based technique is then applied. This technique is based on providing fault tolerance for individual transistors with high detection probability based on transistor duplication and sizing. Experimental results show that the proposed integrated fault tolerance technique achieves similar reliability in comparison to applying STR alone with lower area overhead.

1. Introduction

Due to continuous improvement in technology, systems will not only be increasingly accompanied by manufacturing defects but also susceptible to soft errors during chip operation [1,2]. Reliability of a circuit can be defined as its ability to function properly despite the existence of such errors. Soft errors can arise due to high-energy particles, coupling, power supply noise, leakage and temporal circuit variations. A soft error leads to transient error(s) which can last for one or several clock cycles. A single event transient (SET) occurs when a charged particle hits the combinational logic resulting in a transient current pulse. This can result in an erroneous value at a gate output if the transient has enough width and magnitude. A single event transient becomes a single event upset (SEU) if the erroneous value is latched at a memory element.

Reliability in systems can be achieved by adding redundancy. Redundancy can be added at the module level, gate level, transistor level or at the software level. One of the well known redundancy based

fault tolerance techniques is Triple Modular Redundancy (TMR). It is based on creating three identical copies of the module to be protected and combining their outputs using a majority voter [3]. The TMR technique has been enhanced by Mohanram et al. [4] by triplicating only those nodes with the highest soft error susceptibility. The rest of the nodes are clustered and shared among the triplicated logic. The TMR technique has also been generalized in [5] by protecting the most occurring values at a single or a pair of outputs. A similar idea has been proposed in [6] for designing fault tolerant sequential circuits by protecting states with high probability of occurrence based on adding equivalent states to the protected states with a hamming distance of one. Another technique based on TMR technique maintains a history index of correct computation (HICC) module to select the correct result [7]. Teifel [8] proposed a Double/Dual Modular Redundancy (DMR) fault tolerance technique based on self-voting circuits.

Soft error protection of combinational logic can also be achieved by adding redundancy at the transistor level. Nicolaidis [9] proposed a scheme where all gates in a circuit are duplicated except the last stage

* Corresponding author.

E-mail addresses: atsheikh@kfupm.edu.sa (A.T. Sheikh), aimane@kfupm.edu.sa (A.H. El-Maleh).

gates. The last stage gates are implemented by duplicating each transistor and connecting the duplicate transistors in series with each transistor fed by a different copy of the duplicate gates. This makes the last stage gates act as state preserving gates in case the two duplicate inputs feeding each transistor pair are different. Selective hardening techniques, as the name suggests, protect only the most sensitive gates of the circuit [10,11]. Zhou [12] proposed a gate sizing method that protects the sensitive gates by symmetrically sizing their n_{mos} and p_{mos} transistors. Lazzari [13] proposed an asymmetric transistor sizing technique, i.e., n_{mos} and p_{mos} transistors are sized independently of each other for the most sensitive gates of the circuit, but protecting only transistors directly connected to the output of a gate. Sheikh [14] proposed a selective-transistor redundancy technique protecting only sensitive transistors rather than protecting sensitive gates based on transistor sizing and duplication.

There has been a growing interest to investigate the gate-level invariant relationships to solve multitude of CAD problems. Alves et al. [15–17] used logic implications for online error detection. If any invariant relation is violated, the checker hardware will detect and signal an error. This resulted in very high fault coverage with small area overhead. Use of logic implications for power optimization is proposed by Bahar et al. [18] in which high-power dissipating nodes are eliminated to reduce the switching activity. This results in the reduction of power dissipation of the entire circuit.

Recently, logic implications have been used to enhance soft error tolerance in circuits. The mechanism is simple: if a SET occurs and distorts a signal, then the added functionally redundant wire (FRW) will mask the error from propagating across the gate to a primary output. Almukhaizim et al. [19] proposed a gate-level soft error mitigation technique by adding functionally redundant wires. The proposed method first finds the implication relation between a set of sensitive wires with high detection probability and all other wires in the circuit. Then, for each identified implication relation between a wire and a sensitive wire, a FRW is added to the gate of the sensitive wire and the impact on the soft error rate (SER) of the entire circuit is computed by simulation. Then, the best implication that results in the largest reduction in SER is added. The process is repeated for all sensitive wires in the circuit. This approach has the disadvantage of requiring high computation time as the impact of each implication on SER is done by simulation. Zhou et al. [20] proposed an improvement to this approach which considered electrical and timing window masking effect besides logical masking. To eliminate the high computation time to quantify the value of an implication as done in [19], the authors proposed to estimate the SER due to adding each implication without simulation. However, no details are given on how an implication path is identified. Furthermore, addition of FRWs due to implications is restricted only to non-inverting implications to avoid adding extra inverters which introduce new sites for potential SET. In [21,22], implications have been utilized by adding FRWs at targeted gates for improving noise-tolerance for sub-threshold circuits.

Our proposed implication-based fault tolerance technique has the advantage of reducing the implication learning time by generating and assessing the value of implications between sensitive target wires and potentially useful source wires. This is in contrast to the approaches in [19,20], which generate and assess implications between target sensitive wires and all wires in the circuit. In addition, it is similar to the technique in [20] in estimating the value of adding a FRW due to an implication without the use of simulation. However, it is based on estimating the impact on fault detection probabilities. We provide a detailed algorithm for identifying an implication path and the gates along the path whose detection probabilities will be impacted by adding a FRW due to an implication. Our proposed algorithm handles both direct and indirect implication relations. We show through examples that adding FRW due to some of the learned implication relations will be of no benefit. This is correctly identified by our proposed implication path identification algorithm. Furthermore, the estimation of the

impact of an implication is enhanced by taking into consideration the fanouts of gates along an implication path and those in the logic cone of the target gate. We also add FRWs due to implications either at the target gate or at its fanout gate allowing both inverting and non-inverting implications without any restriction to the number of FRWs added at any gate.

In this paper, we propose an integrated fault tolerance technique where the proposed implication-based fault tolerance technique is applied first to enhance circuit fault tolerance with minimal area overhead. This is followed by our recently proposed selective transistor redundancy (STR) technique [14], which is based on transistor duplication and sizing to protect sensitive transistors with high detection probability. The STR technique is applied to achieve any required circuit reliability.

The rest of the paper is organized as follows. In Section 2, the motivation of the proposed work is presented, in Section 3 the proposed fault tolerance technique is described, illustrative examples are given in Section 4, experimental results are elaborated in Section 5 and the paper is concluded in Section 6.

2. Motivation

In this section, the basic principles of logic implications are briefly explained. Then, we highlight differences between our proposed technique and other related techniques. These are illustrated based on the examples given in [19].

A logic implication from a source gate S to a target gate T indicates a fine-grained invariant relationship that a logic value assignment at the source gate will always enforce a consistent value assignment at the target gate i.e., $(S = u) \Rightarrow (T = v)$, where $u, v \in (0, 1)$. Such forced relations can be used to mask SETs. To illustrate this, let us consider the example circuit in Fig. 1. For the target gate G_8 , one obvious implication is $(e = 1) \Rightarrow (G_8 = 0)$. This is because when $e=1$, then $G_3 = 1$ and $G_8 = 0$. If a SET changes the value of any gate along the implication path i.e., G_3 or G_8 , then the faulty value will be $G_8 = 1$. However, the fault will be observable at the primary output O_1 only when $G_9 = 1$. Since $(e = 1) \Rightarrow (G_8 = 0)$, this implies that $(G_8 = 1) \Rightarrow (e = 0)$, i.e., $G_8 \subseteq \bar{e}$. Thus, $G_8 \cap \bar{e} = G_8$, and adding an AND gate at the output of G_8 combining G_8 and $\bar{e}$ does not change the function of the circuit. Thus, if an inverted FRW, dotted line in Fig. 1, is added to realize the implication $(e = 1) \Rightarrow (G_8 = 0)$, then the SET on gate G_3 or G_8 will always be masked at gate G_{10} before being latched at the primary output O_1 . Similarly, a fault hitting the added inverter flipping its value from 0 to 1 will also be masked by G_8 and will not propagate at the circuit output. However, when $e=0$ the added FRW does not provide any additional protection.

To reduce the impact on performance, the authors in [20] proposed to limit the addition of FRWs to two wires for any target gate. However,

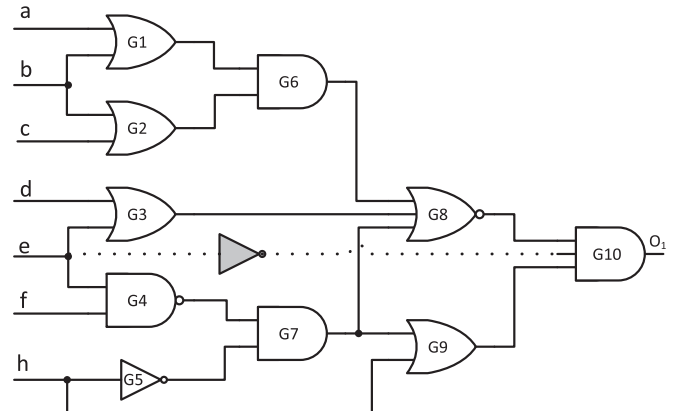


Fig. 1. An example circuit.

this could have a limitation on enhancing circuit fault tolerance. Hence, we do not employ such a restriction. For example, in the circuit of Fig. 1, there are two additional implications to the target gate G_8 i.e., $(b = 1) \Rightarrow (G_8 = 0)$ and $(h = 0) \Rightarrow (G_8 = 0)$. So, realizing these three implications by adding FRWs to the masking gate G_{10} will improve the SET of gates along three different implication paths. A masking gate is a gate that is driven by the target gate. In the hindsight, now the masking gate G_{10} becomes a 5-input AND gate. If there exists a technology limitation or performance impact, then this gate can be split into smaller gates to reduce the delay of the critical path.

Determination of gates along an implication path is not a trivial task. Although the authors in [20] used an implication path in their analysis, no details were given for determining an implication path and the gates along the path. An implication might propagate from the source gate forward to the target gate, might traverse backward then forward or might not have a single path. In Fig. 1, the implication path for the implication $(b = 1) \Rightarrow (G_8 = 0)$, is $b \rightarrow \{G_1, G_2\} \rightarrow G_6 \rightarrow G_8$. However, the implication path for the implication $(G_4 = 0) \Rightarrow (G_8 = 0)$, is $G_4 \rightarrow e \rightarrow G_3 \rightarrow G_8$. Furthermore, the implication $(h = 0) \Rightarrow (G_8 = 0)$ does not have a single implication path. When $h=0$, $G_5 = 1$ and if $G_4 = 1$ this leads to having $G_7 = 1$ and $G_8 = 0$. However, when $G_4 = 0$, this implies that $e=1$, $G_3 = 1$ and $G_8 = 0$.

In this work, we propose an algorithm for identifying an implication path and the gates whose detection probabilities are impacted due to adding a FRW based on a given implication. Addition of FRWs improves the reliability of logic circuits by increasing logical masking. However, the masking gates will remain unprotected and other techniques need to be employed to enhance their fault tolerance. Furthermore, fault tolerance enhancement is constrained by the existence of such relations. In order to further improve circuit reliability, we employ the selective transistor sizing (STR) technique [14], which is based on transistor duplication and sizing to provide protection of sensitive transistors with high detection probability. The combined application of the proposed implication based fault tolerance technique and selective transistor sizing technique [14] requires lesser area overhead as compared to applying STR alone.

3. Proposed fault tolerance technique

The proposed fault tolerance technique is based on the integration of an implication based fault tolerance technique and selective transistor fault tolerance technique. We will first briefly discuss our recently proposed selective-transistor redundancy (STR) technique [14]. Then, the proposed implication based fault tolerance technique is presented.

3.1. Selective transistor redundancy based fault tolerance technique

A recently proposed transistor-level fault tolerance technique [14] protects individual sensitive transistors of a circuit. A sensitive transistor is a transistor whose soft error detection probability is relatively high. A transistor is protected if a required criteria such as circuit probability of failure or area overhead is not met and the probability of failure of the transistor is greater than the probability of failure of other transistors in the circuit. The protection is applied to a transistor by duplicating and sizing one or more transistors necessary for providing the protection against particle strikes. This approach is effective in achieving a target circuit reliability with lower area overhead as compared to other related fault tolerance techniques.

3.2. Implication-based fault tolerance technique

The proposed implication-based fault tolerance technique is shown in Algorithm 1. Initially, the circuit is simulated with 1 million random input patterns using HOPE [23] to get the probability of having a value of 1 and the probability of having a value of 0 and stuck-at (i.e., stuck-at-0 and stuck-at-1) fault detection probabilities for all gates in the circuit.

Next, the set of source gates S and the set of target gates T are identified. To reduce the computation time and to focus on identification of potentially useful implications, only source gates that have probability of output value i.e., probability of zero (P0) or probability of one (P1), greater than or equal to $Th1$ are selected. Thus, $Th1$ defines the minimum probability of having a value of 1 or 0 on a source gate. It should be noted that the higher the source gate probability, the higher the failure rate reduction is when a FRW is added from the source gate to the target gate. Based on extensive evaluation of gains achieved due to adding a FRW based on an implication, it has been observed that source gates that have P0 or P1 ≥ 0.3 are good candidates for implication FRW addition. Thus, $Th1$ has been set to 0.3. $Th2$ is defined as the minimum fault detection probability for a gate to be considered for protection. Thus, target gates that have fault detection probability $\geq Th2$ are selected. From simulations, a good value of $Th2$ is found to be 0.4. It is worth mentioning that these threshold values have been found by exploring all implication relations without the use of any threshold values and then performing analysis on the implications who have the highest reduction in fault detection probabilities along an implication path. Based on this analysis, it was found that the selected threshold values provide reduction in implication learning time with minimal impact on failure rate reductions. Then, implications are identified between gates in S and T using any implication learning technique such as direct implications learning [24] and indirect implications learning [25–27].

Once implications are discovered, Algorithm 1 then evaluates each implication by identifying the implication path for each implication and computing the gain in the form of reduction of gate stuck-at fault detection probabilities for gates along the implication path. Then, the implication with the highest gain is selected and a FRW based on the implication is added. The gain of all remaining implications is then updated by updating fault detection probabilities of all gates along the implication path. The process of implication FRW addition is repeated until the best implication gain $Gain < 0.02$. The gain threshold value of 0.02 has been found based on applying the proposed integration method targeting a given reliability measure by performing simulations of all circuits for different threshold values and evaluating the area overhead needed to achieve the desired reliability. A Threshold of 0.02 offered the lowest overall area overhead as will be illustrated in Section 5. The details of each step of Algorithm 1 are elaborated in the following subsections.

Algorithm 1. : Implications Based Fault Tolerance Technique.

- 1: $Th1$: Gate probability threshold
- 2: $Th2$: Gate Stuck-at fault detection probability threshold
- 3: Φ : Set of implications b/w S and T
- 4: $G_{s_i} \rightarrow G_{t_i}$: Gates along the i^{th} implication path
- 5: $Gain$: Path gain due to implication FRW addition
- 6:
- 7: Simulate circuit to get initial gate value and stuck-at fault detection probabilities
- 8: Identify set of source gates, S , with $Prob(s_{i0})$ or $Prob(s_{i1}) \geq Th1$
- 9: Identify set of target gates, T , with $Pdet(t_i) \geq Th2$
- 10: Extract implication relations between gates in S and T
- 11: //Implications evaluation
- 12: **for**(each implication $\phi_i \in \Phi$) **do**
- 13: Identify implication path $\triangleright$ Algorithms 4 & 5
- 14: Compute implication $Gain \triangleright$ Eq. (3)
- 15: **end for**
- 16: //End of Implications evaluation block
- 17: **repeat**
- 18: //Implications addition
- 19: Select implication ϕ_i with highest gain
- 20: **if**($Gain \geq 0.02$) **then**

```

21:      Add FRW for implication  $\phi_i \triangleright$  Algorithms 2 & 3
22:      Update stuck-at fault detection prob.  $G_{s_i} \rightarrow G_{t_i}$ 
23:      Update Gain for all remaining implications
24:    end if
25:    //End of Implications addition block
26: until(Gain < 0.02)

```

3.2.1. Implications learning

There are two main methods to identify implications in a circuit and any one of them can be employed in our technique. *Direct implications* are identified by assigning a value at a gate and iteratively performing backward justification and forward propagation until every unjustified gate is either justified, or there exist more than one possible justification for it. FAN [24] is a well know algorithm to discover direct implications in a circuit. The identification of *indirect implications* is much harder than the direct implications and implications are identified through learning by injecting temporary values at certain gates in the circuit and then examining their logical consequences [25–27].

3.2.2. Rules for addition of an implication FRW

Once an implication is selected, a FRW is added between the source and the destination gate. Previous methods of implications FRWs addition [19,20] only add FRW to the masking gate. A masking gate is a gate that is connected to the output of the target gate of an implication. In the proposed technique, a FRW can be added to either the masking gate or the target gate. When an implication is added to the masking gate, the masking gate is considered as the new target gate. Algorithm 2 and Algorithm 3 illustrate all the rules required to add a FRW due to an implication $(S = u) \Rightarrow (T = v)$.

Algorithm 2. : FRW Addition Rule for $(S = u) \Rightarrow (T = 1)$.

```

1:  M: Masking gate;  $M_i$ : Set of inputs of Masking gate
2:  T: Target gate;  $T_i$ : Set of inputs of Target gate
3:   $\hat{M}$ : New masking gate;  $\hat{T}$ : New target gate
4:   $FO_T$ : Number of fanouts of target gate
5:   $\oplus$ : Logic XOR
6:
7:  if(M=AND/NAND or  $FO_T > 1$  or  $M=\phi$ ) then
8:    if(T=NOT) then
9:       $\hat{T} \leftarrow \text{NAND}(T_i, (S \oplus u))$ 
10:   else if(T=NAND) then
11:      $\hat{T} \leftarrow \text{NAND}(T_i, (S \oplus u))$ 
12:   else if(T=OR) then
13:      $\hat{T} \leftarrow \text{OR}(T_i, (S \oplus \bar{u}))$ 
14:   end if
15: else
16:   if(M=NOT) then
17:      $\hat{M} \leftarrow \text{NOR}(M_i, (S \oplus \bar{u}))$ 
18:   else if(M=OR) then
19:      $\hat{M} \leftarrow \text{OR}(M_i, (S \oplus \bar{u}))$ 
20:   else if(M=NOR) then
21:      $\hat{M} \leftarrow \text{NOR}(M_i, (S \oplus \bar{u}))$ 
22:   end if
23: end if

```

Algorithm 2 illustrates the implication FRW addition rules when a source value u implies a target value $v=1$. First, the algorithm checks whether the FRW can be added to the masking gate or not. The FRW cannot be added to the masking gate if the target gate is connected to an output (i.e., there is no masking gate) or if the masking gate is of type AND/NAND. In addition, we do not add a FRW to a masking gate

if the fanout of the target gate is > 1 . This is because soft errors on the target gate and other gates on the implication path may propagate across other fanout branches. If the FRW cannot be added to the masking gate, then an attempt is made to add the FRW to the target gate. If the target gate is of type NOT, then it will be changed into a NAND gate with the input of T (i.e., T_i) and the input $S \oplus u$. The XOR operator $\oplus$ performs the inversion of the source gate S when required. For example, if $S = u = 1$ and $T=1$ and the target gate is an inverter, then the FRW from source S must first be inverted before being added to the NAND gate. The relation $(S \oplus u)$ will become $S \oplus 1 \Rightarrow \bar{S}$, which satisfies the condition for adding the FRW from source gate S to target gate T . Similarly, when the target is a NAND gate, then an extra input with an inverted or non-inverted source gate $(S \oplus u)$ is added to the target NAND gate. Also, if the target gate is an OR gate, then an extra input with an inverted or non-inverted source gate $(S \oplus \bar{u})$ is added to the target OR gate.

However, if the FRW can be added to the masking gate, then it is added as follows. For $T=1$, the FRW can only be added to the masking gate if it is of type NOT, OR or NOR. Other cases will require the insertion of an OR gate, which is avoided in this work. If the masking gate is a NOT gate, then it is converted to a NOR gate with new inputs M_i (i.e., “ T ”) and $(S \oplus \bar{u})$. If the masking gate is of type OR or NOR gate, then the inverted or non-inverted source gate $(S \oplus \bar{u})$ is just added to the masking gate as an extra input.

Algorithm 3. : FRW Addition Rule for $(S = u) \Rightarrow (T = 0)$.

```

1:  M: Masking gate;  $M_i$ : Set of inputs of Masking gate
2:  T: Target gate;  $T_i$ : Set of inputs of Target gate
3:   $\hat{M}$ : New masking gate;  $\hat{T}$ : New target gate
4:   $FO_T$ : Number of fanouts of target gate
5:   $\oplus$ : Logic XOR
6:
7:  if(M=OR/NOR or  $FO_T > 1$  or  $M=\phi$ ) then
8:    if(T=NOT) then
9:       $\hat{T} \leftarrow \text{NOR}(T_i, (S \oplus \bar{u}))$ 
10:   else if(T=NOR) then
11:      $\hat{T} \leftarrow \text{NOR}(T_i, (S \oplus \bar{u}))$ 
12:   else if(T=AND) then
13:      $\hat{T} \leftarrow \text{AND}(T_i, (S \oplus u))$ 
14:   end if
15: else
16:   if(M=NOT) then
17:      $\hat{M} \leftarrow \text{NAND}(M_i, (S \oplus u))$ 
18:   else if(M=AND) then
19:      $\hat{M} \leftarrow \text{AND}(M_i, (S \oplus u))$ 
20:   else if(M=NAND) then
21:      $\hat{M} \leftarrow \text{NAND}(M_i, (S \oplus u))$ 
22:   end if
23: end if

```

Algorithm 3 illustrates the FRW addition rules when a source value u implies a target value of $v=0$. The discussion regarding Algorithm 2 can be well extended for the rules given in Algorithm 3.

3.2.3. Implication path identification

Once an implication is identified, the next important step is to evaluate the impact of adding a FRW due to the implication. Almukhaizim et al. [19] used simulation to quantify the impact on soft error rate (SER) when an implication FRW is added. Although they have tried to reduce the number of simulated input values, performing simulation to evaluate the value of each implication is a very time consuming process and will not scale for large circuits. Rather than simulating the circuit to evaluate the value of adding an implication

FRW, Zhou et al. [20] employed a relation to estimate the SER of gates along an implication path. However, they didn't elaborate on how an implication path is identified and how to determine when gates in an implication are impacted by adding a FRW due to an implication. We will show next the details of how an implication path is determined and the gates along the implication path whose fault detection probabilities are reduced when an implication FRW is added.

Two algorithms are proposed to determine an implication path. The first algorithm, value propagation (VP), traverses gates and marks them from source gate S to target gate T . Traversing gates for direct implications can be done based on forward-backward value propagation until the target gate is reached. However, for indirect implications the process is not straightforward. This is because the implied value might propagate across different paths to the target gate. Thus, the set of gates whose fault detection probabilities will be reduced by adding a FRW could be different sets with each set partially protected. Our algorithm attempts to identify such implication paths by propagating values across gates whether they are controlling or non-controlling. For example, a single value at an input of a gate is propagated to its output irrespective of being a controlling or non-controlling value. A marking is used that determines whether a gate has propagated across a gate based on a controlling value at the input of a gate or non-controlling values at all inputs of the gate, denoted as P , or it has propagated based on having non-controlling values but not at all gate inputs, denoted as NP . Then, the algorithm starts by marking the source and performing forward-backward propagation of values until the target gate is reached. This mechanism allows identification of implications paths for direct and indirect implication relations. However, for indirect implications who have alternate implication paths not all paths might be identified.

Once the gates from the source gate to the target gate are marked, the implication path identification (IPI) algorithm then determines the implication path. This is done by starting from the target gate and tracing backward until the source gate is reached. This is done based on following inputs with P markings if a single gate input has such a marking or following NP markings with controlling value. The VP and IPI algorithms are illustrated in Algorithms 4 and 5, respectively.

Algorithm 4. : Value Propagation (VP) Algorithm.

```

1:  $P_0$ : Propagating 0;  $P_1$ : Propagating 1;
2:  $NP_0$ : Non-propagating 0;  $NP_1$ : Non-propagating 1;
3:  $C$ : Controlling value;  $NC$ : Non-controlling value
4:  $IP$ : Set to 1 when a gate is inverting (i.e., NOT, NAND, NOR),
   otherwise set to 0
5:  $processQ$ : Queue to process gates
6:  $CG$ : Current Gate being processed
7:  $u$ : value of  $CG$ 
8:  $M_{CG}$ : Marking applied to the output of  $CG$ 
9:
10: Mark all gates in the logic cone of target gate  $T$ 
11: Mark all gates in the logic cone of source gate  $S$ 
12:  $CG$ =Source gate
13:  $M_{CG}=P_u$ 
14: if( $u \oplus IP = NC$ ) then
15:   Mark all inputs of  $CG$  with the value  $P_{u \oplus IP}$ 
16:   Add all inputs of  $CG$  in the logic cone of  $T$  to  $processQ$ 
17: end if
18: Add all fanouts of  $CG$  in the logic cone of  $T$  to  $processQ$ 
19: while( $processQ \neq \text{NULL}$ ) then
20:    $CG = \text{pop}(processQ)$ 
21:   if(inputs of  $CG$  are not marked and  $u \oplus IP = NC$ ) then
22:     Mark all inputs of  $CG$  with the value  $P_{u \oplus IP}$ 
23:     Add all inputs of  $CG$  in the logic cone of  $T$  to  $processQ$ 
24:   else if(any input to  $CG=P_C$ ) then

```

```

25:    $M_{CG} \leftarrow P_{C \oplus IP}$ 
26:   else if(all inputs to  $CG=P_{NC}$ ) then
27:      $M_{CG} \leftarrow P_{NC \oplus IP}$ 
28:   else if(any input to  $CG=P_{NC}$ ) then
29:      $M_{CG} \leftarrow NP_{NC \oplus IP}$ 
30:   else if(any input to  $CG=NP_C$ ) then
31:      $M_{CG} \leftarrow NP_{C \oplus IP}$ 
32:   else if(all inputs to  $CG=NP_{NC}$ ) then
33:      $M_{CG} \leftarrow NP_{NC \oplus IP}$ 
34:   end if
35:   if(New marking of  $CG \neq$  It's Previous marking) then
36:     add fanouts of  $CG$  in the logic cone of  $T$  and not in the logic
       cone of  $S$  into  $processQ$ 
37:   end if
38: end while

```

(a) *Value Propagation (VP) Algorithm* Algorithm 4 illustrates the steps of the VP algorithm. C and NC denote that for the current gate (CG) under consideration, an input of the gate is having either a controlling or non-controlling logic value. For example, an input value equal to 0 is a controlling value to an AND/NAND gate, whereas a value equal to 1 is a non-controlling input value. The propagating P and non-propagating NP markings hold information regarding the probable implication path. IP indicates the inversion polarity for the current gate which states whether the gate implements inverting or non-inverting logic. IP is 1 for NOT, NAND and NOR gates, while 0 for AND and OR gates. Algorithm 4 starts by identifying all gates in the logic cone of the target gate T , and all gates in the logic cone of the source gate S . Then, the output of the source gate is marked with the value P_u . If the value $u \oplus IP$ is a non-controlling value for the source gate, then all its inputs are marked with $P_{u \oplus IP}$. Then, all marked inputs of the source gate in the logic cone of the target gate are added to $processQ$. Furthermore, all gates in the fanout of the source gate which are in the logic cone of the target gate are added to $processQ$. The algorithm then processes gates in $processQ$ until the queue becomes empty.

Next, a gate, CG , is selected from $processQ$. If inputs of CG are not marked and $u \oplus IP = NC$, then all inputs of CG are marked with the value $P_{u \oplus IP}$ and inputs in the logic cone of the target gate are added to $processQ$. Otherwise, if any of the inputs of the CG is propagating with controlling value, the gate will be marked with $P_{C \oplus IP}$. For example, if one of the inputs of an OR gate is marked with P_1 , then the gate will be marked with P_1 . However, for a NOR gate, if one of the inputs is marked with P_1 , then the gate will be marked with P_0 . If all inputs to a gate are propagating P with non-controlling value NC , then it will be marked with $P_{NC \oplus IP}$. The path will switch from P to NP if none of the inputs of the CG has a P_C marking and not all inputs have P_{NC} markings. Once the implication path becomes NP , it will remain NP . If the marking computed for CG is different from its previous marking, fanouts of CG in the logic cone of T and not in the logic cone of S will be added into $processQ$.

(b) *Implication Path Identification (IPI) Algorithm* The implication path identification (IPI) algorithm is applied after VP algorithm. The purpose of IPI algorithm is to determine the implication path and the gates along the path whose detection probabilities will be impacted by adding the implication FRW. Algorithm 5 highlights the steps of the IPI algorithm.

Algorithm 5. : Implication Path Identification (IPI) Algorithm.

```

1:  $P_0$ : Propagating 0;  $P_1$ : Propagating 1;
2:  $NP_0$ : Non-propagating 0;  $NP_1$ : Non-propagating 1;
3:  $C$ : Controlling value;  $NC$ : Non-controlling value
4:  $PI$ : Primary input
5:  $processQ$ : Queue to process gates

```

```

6:  CG: Current Gate being processed
7:  PathG: Implication path gates
8:
9:  Add T to processQ
10: while(processQ ≠ NULL) do
11:   CG = pop(processQ)
12:   if(CG ≠ S and CG ≠ T) then
13:    Add CG to PathG
14:   end if
15:   if(One input of CG is marked with PC) then
16:    if(Input marked with PC is not PI and ∉ processQ) then
17:     Add input to processQ
18:    end if
19:   else if(All inputs of CG are marked with PNC) then
20:    for(all inputs)
21:     if(Input is not PI and ∉ processQ) then
22:      Add input to processQ
23:     end if
24:    end for
25:   else if(One input of CG is marked with NPC) then
26:    if(Input marked with NPC is not PI and ∉ processQ) then
27:     Add input to processQ
28:    end if
29:   else if(All inputs of CG are marked with NPNC) then
30:    for(all inputs) do
31:     if(Input is not PI and ∉ processQ) then
32:      Add input to processQ
33:     end if
34:    end for
35:   end if
36: end while
37: Return PathG

```

The *processQ* holds the list of gates to be processed in each iteration, whereas *Path_G* list contains gates that are part of the implication path. The set of rules established in Algorithm 5 will now be explained. The algorithm starts with adding the target gate to *processQ*. If only one input to a gate is marked with *P_C*, then that input is added to the *processQ* and the process continues. It should be observed that if two or more inputs of a gate are marked with *P_C*, then the inputs of this gate are not added to *processQ*. Fig. 2 illustrates this case. For both circuits, the value of *A*=0 or *A*=1 is propagated to the output. Therefore, if 2 or more inputs of the target gate are marked with *P_C*, then there is no need to protect the gates along the implication path(s) as faults on gates along one path will be masked at the target gate due to the controlling logic value of the other input.

Fig. 3 illustrates the application of rule 2. If all inputs of the target gate are marked with *P_{NC}*, then all the inputs are added to the *Path_G* list. Therefore, both *G1* and *G2* are added to the *Path_G* list in this case. Other rules will be explained through illustrative examples given in the next section.

3.2.4. Evaluation of implication FRW gain

Once the implication path is determined, the next step is to evaluate the value of adding a FRW due to an implication path based on updating the stuck-at fault detection probabilities of gates along the

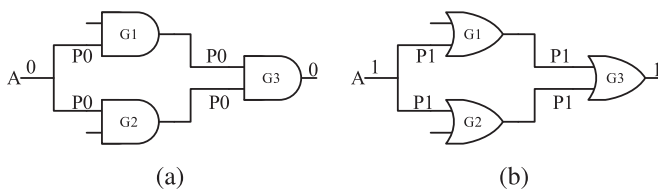


Fig. 2. Illustration of having two inputs with *P_C* markings.

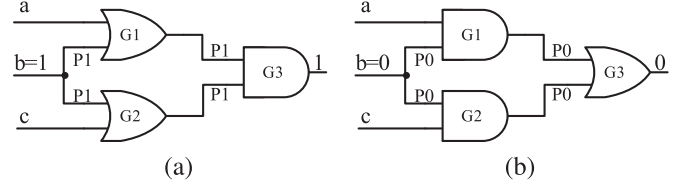


Fig. 3. Illustration of having all inputs with markings *P_{NC}*.

implication path. If the value *v* of a gate *G_i* along an implication path is “0”, then the stuck-at-1 (sa1) fault detection probability of *G_i* is reduced due to adding the implication FRW. However, if the value of the gate is “1”, then the stuck-at-0 (sa0) fault detection probability of gate *G_i* is reduced. The following equation is used to update or modify the stuck-at fault detection probability of gate *G_i* along an implication path.

$$\widehat{G}_{i_{sa\bar{v}}} = G_{i_{sa\bar{v}}} \times (1 - P_u) \times PRP \quad (1)$$

Where *P_u* denotes the probability that the source gate *S* has a value of *u* and *PRP* is the percentage of paths from gate *G_i* in the logic cone of the target gate *T* computed using the following relation:

$$PRP = \frac{\# \text{ of paths } G_i \rightarrow T}{\# \text{ of paths } G_i \rightarrow PO} \quad (2)$$

Eq. (2) is used to estimate the percentage reduction in the fault detection probability of gate *G_i* as fault detection will be reduced for paths propagating through *T* but not across other paths not propagating through *T*.

For each implication, Eq. (1) is applied to update the stuck-at fault detection probabilities of gates along the implication path. The gain of an implication is then computed using Eq. (3).

$$Gain_{S \rightarrow T} = \sum_{\forall G_i \in S \rightarrow T} G_{i_{sa}} - \sum_{\forall G_i \in S \rightarrow T} \widehat{G}_{i_{sa}} \quad (3)$$

Where $\widehat{G}_{i_{sa}}$ denotes the new fault detection probability and *G_{i_{sa}}*

 denotes the old fault detection probability of gate *G_i*.

4. Illustrative examples

In this section, two examples are discussed in detail that encompass all aspects of the proposed method discussed in Section 3. Consider the example shown in Fig. 4, where an implication exists between source gate *G4* and the target gate *G3*. The masking gate in this example is gate *G10*. The first step is to determine whether the FRW will be added to the target gate *G3* or the masking gate. Since the implication is of type (*G4* = 0) $\Rightarrow$ (*G3* = 0) and the masking gate *G10* is an AND gate, therefore the FRW will be added to the masking gate. The new target gate *T* is now *G10*. The circuit with the added FRW (dotted line) is also shown in Fig. 4. The masking gate *G10* now becomes a 3-input AND gate with an extra input connected to the source gate *G4*.

Once the FRW is added, the next step is to determine the implication path. Starting from the source gate, *G4* is marked as

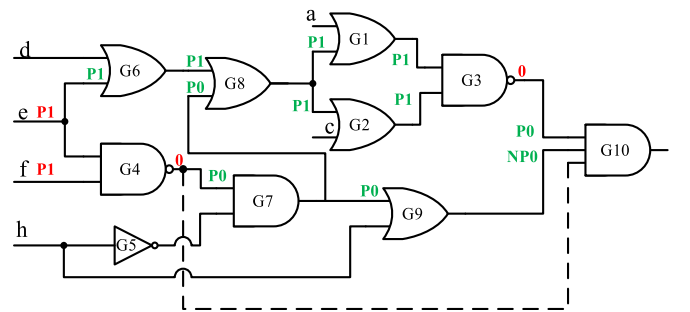


Fig. 4. Circuit with Implication (*G4* = 0) $\Rightarrow$ (*G3* = 0).

propagating P with value 0 i.e., P_0 . Since source gate $G4$ is a NAND gate, the value of IP is equal to “1”. The condition in line 14 of Algorithm 4 i.e., $(0 \oplus 1 = 1(NC))$, is satisfied, therefore all inputs of $G4$ are marked with $P_{0\oplus 1} = P_1$. Primary inputs e and f are also added to the *processQ* as they are both in the logic cone of the target $G10$. So, initially, $G4$ is marked as P_0 and both primary inputs e and f are marked as P_1 . $G4$ has only one fanout gate $G7$, which is in the logic cone of the target $G10$, therefore it is also added to the *processQ* due to the condition in line 18 of Algorithm 4. The *processQ* now contains values $\{e, f, G7\}$.

Next, e is selected as CG from the *processQ*. Since e is a primary input, all the conditions from line 21 to line 32 are skipped. However, due to the statement in line 35 of Algorithm 4, $G6$ is added to the *processQ*, because it is the only fanout of e in the logic cone of the target gate $G10$. The other fanout of e is the source $G4$, therefore it is not added to the *processQ*. The *processQ* now consists of $\{f, G7, G6\}$. In the next iteration, f is selected as CG . Again, all marking conditions are skipped as f is a primary input. Since the only fanout for f is the source $G4$, nothing is added to the *processQ*. So, the *processQ* now contains $\{G7, G6\}$. In the next iteration, $G7$ is selected as CG . Since one of the inputs of $G7$ is P_0 , i.e., satisfying the condition in line 24 of Algorithm 4, $G7$ is marked as P_0 i.e., $M_{G7} = P_{C=0\oplus IP=0} = P_0$. Both fanouts of $G7$ are added to the *processQ* as they are in the logic cone of the target gate $G10$. The updated *processQ* now consists of $\{G6, G8, G9\}$.

Gate $G6$ is marked as P_1 as its input is marked with P_1 . Gate $G8$, which is the fanout of $G6$ is now added to the *processQ*. Updated *processQ* is now $\{G8, G9, G8\}$. The inputs to $G8$ are marked as P_1 due to $G6$ and P_0 due to $G7$, therefore due to the condition in line 24 of Algorithm 4, $G8$ is marked as P_1 . After both fanouts of $G8$ are added, the updated *processQ* becomes $\{G9, G8, G1, G2\}$. One of the inputs to gate $G9$ is marked as P_0 due to $G7$, therefore, due to the condition in line 28 of Algorithm 4, the output of $G9$ is marked as non-propagating NP with value 0 i.e., $M_{G9} = NP_{NC=0\oplus IP=0} = NP_0$. Nothing is added to the *processQ* as the fanout of $G9$ is the target gate $G10$. In the next iteration, $G8$ is selected again but its marking will stay the same i.e., $M_{G8} = P_1$. Thus, its fanouts $G1$ and $G2$ are not added to the *processQ*. Both $G1$ and $G2$ have inputs marked with P_1 , therefore both of them are marked as P_1 . $G3$ is now added to the *processQ*. Finally, $CG = G3$ is selected for processing. All inputs of $G3$ are P_1 i.e., propagating with non-controlling value, therefore, due to the condition in line 26 of Algorithm 4, $G3$ is marked as P_0 i.e., $M_{G3} = P_{NC=1\oplus IP=1} = P_0$. Nothing is added to the *processQ* as the fanout of $G3$ is the target $G10$. The algorithm terminates as the *processQ* is now empty. The gate markings due to VP algorithm are shown in Fig. 4.

Once the VP algorithm is finished, the IPI Algorithm (Algorithm 5) is applied to determine the implication path and the gates protected by adding the implication FRW. The IPI algorithm starts from the target $G10$. Starting from $G10$, $G3$ is selected and is added to the *processQ*, as it has $P0$ marking, in accordance with the rule mentioned in line 15 of Algorithm 5. Then, $G3$ is processed and is added to *Path_G*. Since both inputs of $G3$ are marked with P_1 , P with non-controlling input $NC=1$, therefore both $G1$ and $G2$ are added to *processQ* in accordance with the rule mentioned in line 19 of Algorithm 5. Then, $G1$ and $G2$ get processed and get added to *Path_G* and $G8$ gets added to *processQ*. Then, gate $G8$ is processed and is added to *Path_G*. Since $G8$ has two inputs with markings $P1$ and $P0$, the path with the propagating controlling value is selected i.e., $G6$. Finally $G6$ is processed and is added to *Path_G*. The algorithm now terminates as the input to $G6$ is a primary input. The implication path now consists of gates $(G6 \rightarrow G8 \rightarrow (G1, G2) \rightarrow G3)$.

Once the implication path gates are identified, the next step is to update the stuck-at fault detection probabilities of those gates using Eq. (1). So, for gates along the implication path, $G6sa0$, $G8sa0$, $G1sa0$, $G2sa0$ and $G3sa1$ fault detection probabilities could be reduced. Gate $G6$ has two paths to the primary output $G10$ i.e., $(G6 \rightarrow G8 \rightarrow G1 \rightarrow G3 \rightarrow G10)$ and $(G6 \rightarrow G8 \rightarrow G2 \rightarrow G3 \rightarrow G10)$

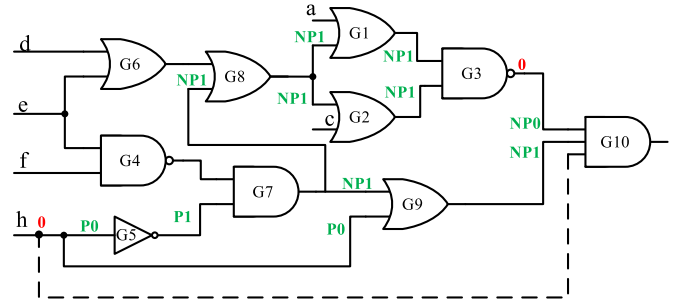


Fig. 5. Circuit with implication ($h = 0$) $\Rightarrow$ ($G3 = 0$).

and all the paths are in the logic cone of the target gate $G10$. Therefore, *PRP* of $G3$ is “1”. In fact, in this example, the *PRP* of all gates along the implication path is “1”. Before adding the implication FRW, the stuck-at fault detection probabilities of gates along the implication path are $\{0.281, 0.563, 0.781, 0.781, 0.781\}$. After the addition of the implication FRW, the estimated stuck-at fault detection probabilities using Eq. (1) along the implication path are $\{0.211, 0.422, 0.586, 0.586, 0.586\}$. Based on actual fault simulation using HOPE [23], the reduced stuck-at fault detection probabilities are found out to be $\{0.188, 0.468, 0.656, 0.656, 0.656\}$. It is clear that the estimated values and the values computed using HOPE [23] are close. The estimated implication path gain computed using Eq. (3) is 0.796 while the path gain obtained using HOPE [23] is 0.563.

Let us next consider the example shown in Fig. 5, where an implication exists between source wire h and the target gate $G3$. The masking gate in this example is gate $G10$. The FRW will be added to the masking gate and the new target gate T is now $G10$. The circuit with FRW (dotted line) is also shown in Fig. 5. Starting from h , *processQ* consists of $\{G5, G9\}$. When selected, $G5$ is marked with P_1 at its input is marked with P_0 . When $G7$ is added, *processQ* becomes $\{G9, G7\}$. Currently, only one input to $G9$ is marked as $P0$ due to h , therefore, $G9$ is marked NP_0 due to condition in line 26 of Algorithm 4. Since the output of $G9$ is target $G10$, nothing is added to the *processQ*. The *processQ* now consists of $\{G7\}$ only. $G7$ is marked with NP_1 as its input is a non-controlling value. Both fanouts of $G7$ are added to the *processQ* and it will become $\{G8, G9\}$. $G8$ is processed next and is marked with NP_1 and its fanouts are added to the queue. *processQ* becomes $\{G9, G1, G2\}$. $G9$ is selected for the second time now and this time both its inputs are marked as NP_1 due to $G7$ and P_0 due to h . The marking of $G9$ changes from NP_0 to NP_1 due to the controlling input value from $G7$. $G1$ and $G2$ are then processed and are also marked with NP_1 . The *processQ* now consists of $\{G3\}$ only. Finally, when selected, $G3$ is marked with NP_0 . VP algorithm now terminates as the target is reached and *processQ* become empty. The markings of gates due to VP algorithm are shown in Fig. 5. It must be noted that when a path becomes non-propagating NP it remain NP until the target is reached.

The IPI algorithm is now applied to determine the implication path. It can be observed from Fig. 5 that both inputs of $G10$ are NP . The only clue for the IPI algorithm now is to take the path with controlling value i.e., condition in line 25 of Algorithm 5. Therefore, $G3$ is selected and then added to the *processQ* and to *Path_G*. All inputs of $G3$ have NC value, therefore, due to condition in line 29 of Algorithm 5, both $G1$ and $G2$ are added to the *processQ* and *Path_G*. When processing $G1$ and $G2$, $G8$ is added to the path gates list *Path_G* and *processQ*. Continuing in this fashion, $G7$ and $G5$ are also added to the *Path_G*. Algorithm 5 now terminates as the source h is reached and also the *processQ* becomes empty. The implication path now consists of gates $(G5 \rightarrow G7 \rightarrow G8 \rightarrow (G1, G2) \rightarrow G3)$. So, for gates along the implication path, $G5sa0$, $G7sa0$, $G8sa0$, $G1sa0$, $G2sa0$ and $G3sa1$ fault detection probabilities could be reduced. Before FRW is added, stuck-at fault detection probabilities of gates along the implication path are $\{0, 0, 0.563, 0.781, 0.781, 0.781\}$. The reduced stuck-at fault

detection probabilities of implication gates computed using HOPE [23] are {0, 0, 0.281, 0.406, 0.406, 0.406}. The estimated stuck-at fault detection probabilities computed using Eq. (1) are {0, 0, 0.281, 0.391, 0.391, 0.391}. The estimated path gain computed using Eq. (3) is 1.452 while the actual gain obtained using HOPE [23] simulation is 1.407. It is clear that the estimated values are very close to the simulated values obtained using HOPE [23].

It should be observed that since this is an indirect implication, identified gates along the implication path will be partially protected. This is because when $G4 = 1$ and $e=0$, all the identified gates by the implication path will be protected by reducing their fault detection probabilities. When $e=1$, fault detection probabilities of gates $G5$ and $G7$ will not be impacted as their fault propagation is already masked by $G6 = 1$. However, when $G4 = 0$, this implies that $G7 = 0$, $G9 = 0$, $e=1$ and $G3 = 0$. Since both $G3$ and $G9$ have a value of 0, adding a FRW due to the implication will not have an impact on the fault detection probability of any gate.

Let us now consider an example to illustrate the benefits of our proposed integrated approach integrating FRW addition based on implications and selective transistor redundancy addition. Consider the circuit shown in Fig. 6a. The probability of failure of the circuit (POF_c) [14] given in Fig. 6a is 0.191 with an area of $6.24\mu m$. The area is computed by summing the nMOS and pMOS drain widths using 130 nm Predictive Technology Model (PTM) [28]. In order to achieve POF_c of 0.02, our recently proposed selective-transistor redundancy (STR) technique [14] is applied to the circuit shown in Fig. 6a by protecting critical transistors until the desired circuit failure rate is achieved. The desired $POF_c = 0.02$ of the circuit is achieved with an area overhead of 213.33% by fully protecting all transistors in the gates $G3$ and Z , as shown in Fig. 7a.

Then, let us consider adding a FRW due to the implication ($A = 0 \Rightarrow G3 = 0$) by adding an extra input A to the masking NAND gate Z , resulting in the circuit shown in Fig. 6b. This results in a significant reduction in POF_c from 0.191 to 0.122 with an area overhead of just 12.5%. Let us then consider applying STR on the circuit shown in Fig. 6b to obtain further reduction in POF_c of the circuit. The desired $POF_c = 0.02$ is achieved by fully protecting all the transistors in gate Z with an area overhead of 165.83% compared to the original circuit, as shown Fig. 7b. Thus, this example illustrates that the integrated approach requires less area overhead to achieve a given POF_c in comparison to applying STR alone.

5. Experimental results

In this section, we evaluate the impact of the proposed technique on the area and reliability of LGSynth'91 [29] benchmarks, which consist of circuits with varying complexity in terms of area, number of inputs and outputs. The LGSynth'91 benchmark circuits used in this work are synthesized with single output optimization using Espresso [30] tool and then mapped to a library that consists of an Inverter and 2-, 3- and 4-input NAND and NOR gates using SIS [31] tool to get the proper gate level representation of the circuit. The reliability of a circuit is computed using the method proposed by Sheikh et al. [14].

The reliability of a circuit is computed against a single fault and the prorated 1 and 2 faults for each circuit. The number of prorated faults is correlated to the area of a circuit. So, for example, if the area overhead is 131%, then the actual area is increased by 2.31 times. Therefore, 1, 2 and 5 faults in the original circuit will prorated to 2.31,

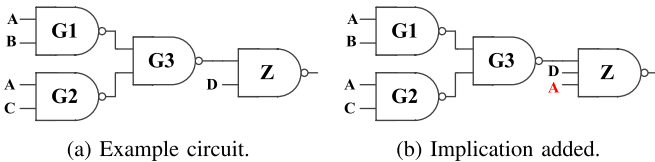


Fig. 6. Implication example. (a) Example circuit. (b) Implication added.

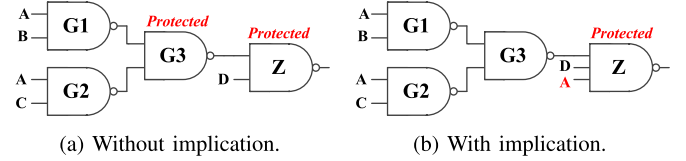


Fig. 7. Protected gates when STR [14] is applied. (a) Without implication. (b) With implication.

4.62 and 11.55 faults in the protected circuit. For each prorated fault, the circuit is simulated twice. For example, if the prorated faults to be injected are 4.62, then the circuit is simulated twice, first by injecting 4 faults and then by injecting 5 faults. The failure rate is then computed as $0.38 * FR(4 \text{ faults}) + 0.62 * FR(5 \text{ faults})$. For each fault injection scenario, faults are injected randomly and simulation is performed for 5000 iterations to compute the failure rate.

The implications are applied to a circuit until an implication gain of a circuit is < 0.02 . Once FRWs due to selected implications are added, the probability of failure of the circuit (POF_c) [14] is then computed. Table 1 shows the circuits area overhead and reliability achieved by the proposed implication based fault tolerance technique. The column header “# Imp.” denotes the number of FRWs due to implications added to each circuit. The column “% Red.” denotes the percentage reduction in the failure rate (FR) of each circuit. This is computed as:

$$\frac{(Original\ FR - FR\ after\ adding\ implications)}{Original\ FR} \times 100$$

The average reliability of all circuits is improved as compared to the original circuits. The average area overhead incurred as a result of implications FRWs addition is 4.58% and the average number of implications utilized is 46.50. Except for *alu4* and *b12*, the POF_c of all the other benchmarks is reduced by at least 14%. The columns “P1” and “P2” show the reliability of circuits against prorated 1 and 2 faults, respectively. It is clear that the average reliability achieved by the proposed technique against prorated faults is better than the reliability of original circuits against 1 and 2 faults.

The proposed implication based fault tolerance technique is based on enhancing logical masking and is applied at the gate level. One limitation of implication-based logical masking techniques is that they are unable to improve the reliability of a circuit beyond a certain point. This is because they are limited by the existence of such implication relations in a circuit. In addition, while adding an implication FRW increases logical masking for a given set of faults on gates across the implication path, it does not have an impact on the corresponding complementary faults on those gates. For example, if the detection probability of a stuck-at-1 fault on a gate along the implication path is reduced due to adding a FRW, the stuck-at-0 fault detection probability is not impacted.

So, in order to further improve the reliability, circuit-level techniques such as gate/transistor resizing have to be employed. Therefore, the reliability of circuits in Table 1 is further enhanced by applying our recently proposed selective-transistor redundancy (STR) technique [14]. Table 2 shows the reliability and area overhead of circuits when STR is applied to the original benchmark circuits for 95%, 98% and 99% protection threshold. Protection threshold refers to the target reliability required to be achieved by a circuit against a single fault. In the proposed integrated approach, the implication based fault tolerance technique is applied first followed by the STR technique if the original circuit does not satisfy a required reliability. The zero area overhead implies that the basic implementation of a circuit is able to achieve the required reliability criteria.

Table 3 shows the circuits reliability and area overhead resulting from the application of the proposed integrated approach for 95%, 98% and 99% protection thresholds. It can be observed from Table 3 that the area overhead is reduced for all protection thresholds in comparison to the application of the STR technique alone. This is due to the

Table 1

Circuits reliability and area overhead based on proposed implication based fault tolerance technique.

Circuit	Area (μm^2) ¹	1 Fault	2 Faults	Proposed Method					
				OH ²	# Imp.	1F ³	% Red.	P1 ⁴	P2 ⁴
alu4	1429.74	97.89%	95.86%	2.73%	41	97.96%	3.32%	97.90%	96.10%
apex1	4602.00	96.72%	94.20%	3.22%	148	97.50%	23.78%	97.32%	95.68%
apex2	609.96	99.20%	98.04%	1.68%	35	99.57%	46.25%	99.50%	98.70%
apex3	3025.62	96.88%	94.76%	5.26%	133	97.32%	14.10%	97.15%	96.10%
apex4	4575.48	96.20%	92.74%	3.03%	135	96.88%	17.89%	96.40%	93.80%
b12	121.68	89.10%	78.22%	12.18%	15	90.10%	9.17%	89.45%	80.22%
clip	372.84	93.24%	86.40%	4.18%	17	94.44%	17.75%	93.97%	86.40%
cordic	241.02	98.10%	96.28%	2.91%	7	98.66%	29.47%	98.20%	96.70%
ex5	977.34	93.50%	88.26%	13.49%	102	94.76%	19.38%	94.11%	90.03%
misex1	152.88	84.34%	71.70%	2.55%	3	88.89%	29.05%	88.37%	73.09%
misex2	230.88	93.20%	88.32%	7.77%	12	94.51%	19.26%	94.10%	89.57%
misex3	1886.82	97.64%	95.40%	3.06%	50	98.61%	41.10%	98.55%	96.89%
rd84	496.08	93.38%	87.22%	5.35%	30	94.69%	19.79%	94.33%	87.61%
seq	4970.94	99.05%	98.22%	0.35%	10	99.20%	15.79%	99.15%	98.76%
squar5	101.40	83.44%	69.46%	5.38%	5	86.78%	20.17%	86.26%	70.36%
table3	3475.68	98.68%	97.78%	0.96%	27	98.89%	15.91%	98.77%	98.11%
table5	3535.74	98.70%	97.72%	1.57%	46	98.91%	16.15%	98.64%	98.22%
z5xp1	251.16	87.96%	75.38%	6.83%	21	90.23%	18.85%	89.45%	75.88%
Avg.		94.29%	89.22%	4.58%	46.50	95.44%	20.96%	95.09%	90.12%

¹ Summation of nMOS and pMOS drain widths.² Area overhead (OH) = $\left(\frac{\text{Area After Implications FRWs Addition}}{\text{Original Area}} - 1 \right) \times 100$.³ Single fault.⁴ Prorated 1 and 2 faults.**Table 2**

Circuits reliability and area overhead based on STR technique [14] with varying protection thresholds against a single fault.

Circuit	95%		98%		99%	
	OH	Rel	OH	Rel	OH	Rel
alu4	0%	97.89%	20.04%	98.44%	51.84%	99.02%
apex1	0%	96.72%	12.64%	98.10%	48.81%	99.10%
apex2	0%	99.20%	0%	99.20%	0%	99.20%
apex3	0%	96.88%	13.93%	98.05%	51.57%	99.08%
apex4	0%	96.20%	25.88%	98.30%	71.88%	99.02%
b12	86.50%	95.01%	149.02%	98.03%	197.69%	99.05%
clip	14.64%	95.00%	68.61%	98.11%	119.87%	99.08%
cordic	0%	98.10%	0%	98.10%	19.53%	99.00%
ex5	19.53%	95.62%	81.73%	98.12%	139.89%	99.02%
misex1	85.68%	95.07%	175.48%	98.01%	249.35%	99.02%
misex2	14.12%	95.24%	50.29%	98.36%	92.48%	99.20%
misex3	0%	97.64%	3.33%	98.10%	34.27%	99.18%
rd84	14.16%	95.10%	56.49%	98.02%	101.21%	99.24%
seq	0%	99.05%	0%	99.05%	0%	99.05%
squar5	107.79%	95.10%	206.15%	98.06%	286.51%	99.00%
table3	0%	98.68%	0%	98.68%	0.34%	99.24%
table5	0%	98.70%	0%	98.70%	1.70%	99.32%
z5xp1	68.22%	95.06%	143.69%	98.11%	202.88%	99.06%
Avg.	22.81%	96.68%	55.96%	98.31%	92.77%	99.11%

fact that the addition of FRWs increases logical masking of faults and hence when STR is applied, fewer transistors require protection to achieve the desired reliability. Significant area savings are achieved for *b12*, *clip*, *ex5* and *z5xp1* benchmarks with 95% threshold as shown in Table 3 in comparison to the results in Table 2. With 98% threshold, *alu4*, *apex1*, *apex3*, *b12*, *ex5*, and *misex2* in Table 3 achieve significantly lower area overhead with similar reliability in comparison to the results in Table 2. Similarly, for 99% threshold, benchmarks such as *alu4*, *apex1*, *apex3*, *apex4*, *b12*, *cordic*, *ex5*, *misex2* and *misex3* have significantly lower area overhead as compared to their counterparts in Table 2, and again with similar reliability measure.

As explained before, the gain threshold used in our algorithm is used as a criteria to determine whether to add a FRW due to an implication or not. This gain threshold has been evaluated based on the

Table 3

Circuits reliability and area overhead based on proposed integrated approach against a single fault.

Circuit	95%		98%		99%	
	OH	Rel	OH	Rel	OH	Rel
alu4	0%	97.89%	7.23%	98.40%	29.01%	99.03%
apex1	0%	96.72%	4.26%	98.05%	31.94%	99.18%
apex2	0%	99.20%	0%	99.20%	0%	99.20%
apex3	0%	96.88%	6.17%	98.1%	37.97%	99.03%
apex4	0%	96.20%	18.19%	98.20%	57.99%	99.01%
b12	23.80%	95.11%	90.30%	98.10%	154.19%	99.09%
clip	8.45%	95.09%	58.88%	98.07%	104.13%	99.06%
cordic	0%	98.10%	0%	98.10%	13.46%	99.1%
ex5	5.26%	95.55%	58.77%	98.1%	107.86%	99.07%
misex1	77.21%	95.12%	169.29%	98.04%	253.54%	99.01%
misex2	11.26%	95.3%	33.90%	98.28%	77.18%	99.15%
misex3	0%	97.64%	1.20%	98.1%	19.49%	99.09%
rd84	16.13%	95.09%	57.31%	98.03%	98.71%	99.04%
seq	0%	99.05%	0%	99.05%	0%	99.05%
squar5	105.73%	95.10%	196.75%	98.01%	285.13%	99.02%
table3	0%	98.68%	0%	98.68%	0.20%	99.12%
table5	0%	98.70%	0%	98.70%	0.46%	99.20%
z5xp1	58.84%	95.11%	129.34%	98.09%	189.46%	99.05%
Avg.	17.04%	96.70%	46.20%	98.29%	81.15%	99.08%

minimum overall area overhead when redundancy-based implication is integrated with selective transistor redundancy for achieving a given reliability measure. Table 4 shows the area overhead obtained for the integrated method for varying gain thresholds. As can be seen from the results, a gain threshold of 0.02 provides the best overall area overhead for achieving a given reliability measure. The gain threshold can be used as parameter with our algorithm to allow the designer to explore different solutions as the best selected gain threshold value could be circuit dependent.

Next, we compare our proposed implication-based fault technique with the techniques proposed by Almukhaizim et al. [19] and Zhou et al. [20]. Comparison is made based on soft error rate (SER) as the results reported in [19,20] are based on SER. Tables 5, 6 show the comparison results. In [19], the area overhead is computed based on

Table 4
Gain analysis for varying threshold values.

Circuit	Gain Th.=0.03			Gain Th.=0.02			Gain Th.=0.01		
	95%	98%	99%	95%	98%	99%	95%	98%	99%
alu4	0%	7.70%	31.45%	0%	7.23%	29.01%	0%	7.85%	29.66%
apex1	0%	2.22%	30.51%	0%	4.26%	29.10%	0%	4.61%	32.92%
apex2	0%	0%	0%	0%	0%	0%	0%	0%	0%
apex3	0%	6.84%	38.88%	0%	6.17%	36.97%	0%	6.49%	38.39%
apex4	0%	18.75%	58.30%	0%	18.19%	56.99%	0%	18.15%	58.66%
b12	23.80%	90.30%	154.19%	23.80%	90.30%	154.19%	23.80%	90.30%	154.19%
clip	9.40%	60.61%	106.20%	8.45%	58.88%	104.13%	12.01%	61.87%	111.81%
cordic	0%	0%	12.82%	0%	0%	13.46%	0%	0%	14.76%
ex5	6.46%	58.06%	108.38%	5.26%	58.77%	107.86%	8.50%	63.28%	113.35%
misex1	77.21%	169.29%	253.54%	77.21%	169.29%	253.54%	77.21%	169.29%	253.54%
misex2	11.82%	35.66%	79.25%	11.26%	33.90%	77.18%	11.26%	33.90%	77.18%
misex3	0%	1.20%	20.15%	0%	1.20%	19.49%	0%	1.20%	19.49%
rd84	16.50%	57.89%	99.12%	16.13%	57.31%	98.71%	18.41%	59.98%	100.27%
seq	0%	0%	0%	0%	0%	0%	0%	0%	0%
squar5	105.73%	196.75%	285.13%	105.73%	196.75%	285.13%	106.15%	202.00%	281.74%
table3	0%	0%	0.20%	0%	0%	0.20%	0%	0%	0.20%
table5	0%	0%	0.46%	0%	0%	0.46%	0%	0%	0.46%
z5xp1	60.22%	130.72%	190.68%	58.84%	129.34%	189.46%	58.84%	129.34%	189.46%
Avg.	17.29%	46.44%	81.63%	17.04%	46.20%	80.88%	17.57%	47.12%	82.01%

the number of transistors, while in [20] the area overhead is computed based on Design Compiler tool from Synopsys using STM 65 nm standard cell library. Therefore, for a fair comparison, the same number of functionally redundant wires added based on implications is used for all compared techniques. The first column in both tables show the benchmark circuits used for comparison along with their number of inputs and outputs. In comparison to the technique proposed by Almukhaizim et al. [19], our proposed technique achieves significantly higher average SER reduction for the same number of implications, as evident from Table 5. The average area, power and delay overhead by the proposed technique are also significantly lower than the one proposed by Almukhaizim et al. [19]. The CPU time of our proposed technique is also significantly lower than the technique in [19] due to the large learning time of all indirect implications and the use of simulations to assess the impact of each implication.

Next, in comparison to the technique proposed by Zhou et al. [20], our proposed technique achieves significantly higher average reduction in SER for the same number of implications, as evident from Table 6. However, the proposed technique incurs slightly higher average area, power and delay overhead in comparison to the one proposed in [20]. This is due to the fact that the technique proposed in [20] uses only non-inverting implications and this results in lower area overhead but this comes at a price of getting lower enhanced reliability. In addition,

the maximum number of added FRWs to a gate is restricted to 2 in [20] to reduce the impact on delay. It should be observed the CPU times of our technique is higher than the ones reported by [20] as our reported CPU times include implication learning time and the time of running our proposed algorithm while the CPU times reported by [20] do not include the time of implication learning, which is the most time consuming step.

Our proposed technique achieves higher reduction in SER than the approaches in [19,20] for all the compared circuits, except circuit s298 in comparison to [20]. This is attributed to several factors including the criteria for selecting the target gates to be protected. For example, the work in [19] sets the target gates to be considered for protection as the most sensitive 25 gates in the circuit, while the work in [20] provides protection for gates which have 60% of the total SER. In our approach, the set of target gates are those who have fault detection probability greater than or equal to 0.4. In addition, the criteria used for assessing the gain due to adding an implication FRW has an impact on the selected implications. For example, the technique in [19] is based on simulating 250 random input patterns to identify the most sensitive target for which an implication is to be added. The use of only 250 random patterns compromises the accuracy of the obtained results. However, the technique in [20] and our proposed technique are based on estimating the gain for each implication. This approach has also

Table 5
Comparison with Almukhaizim [19].

Circuit	AlMukhaizim						Proposed: Same #Implications as [19]				
	#FRW	Time (s)	Red (%)	Overhead (%)			Time (s)	Red (%)	Overhead (%)		
			SER	Area	Power	Delay		SER	Area	Power	Delay
s298 (17, 20)	4	403	6.86	2.05	1.82	0.00	4.87	9.87	2.87	1.39	5.41
s344 (24, 26)	4	427	2.83	2.23	6.07	0.00	138.55	6.61	1.89	3.38	0.37
s386 (13, 13)	10	137	5.53	2.88	5.51	12.30	58.78	36.56	7.10	0.33	0.00
s444 (24, 27)	21	933	7.94	10.80	20.65	14.14	204.08	31.59	9.00	10.39	0.00
s510 (25, 13)	29	4354	16.47	17.92	40.34	38.19	376.78	22.94	15.60	8.99	1.28
s641 (54, 43)	8	931	5.94	2.78	14.14	10.17	2233.14	11.75	5.54	5.25	5.27
s713 (54, 42)	16	1125	12.49	4.74	28.90	23.35	2397.78	19.77	4.35	5.29	2.11
s953 (45, 52)	45	21020	40.45	17.49	21.74	29.70	2695.38	48.85	12.64	6.95	9.92
s1238 (32, 32)	60	319916	35.78	21.35	36.76	37.67	571.36	44.69	14.34	6.14	23.28
s1423 (91, 79)	77	42380	17.09	16.07	29.01	36.62	1049.43	23.18	11.65	9.65	5.74
s1488 (14, 25)	28	19804	7.21	6.99	17.63	12.48	1742.50	40.77	5.73	1.36	6.51
s1494 (14, 25)	65	34507	14.62	17.49	23.87	29.83	1779.79	47.41	12.15	2.40	5.66
Avg.	30.58	37161.42	14.43	10.23	20.54	20.37	1104.37	28.67	8.57	5.13	5.46

Table 6
Comparison with Zhou [20].

Circuit	Zhou						Proposed: Same #Implications as [20]				
	#FRW	Time (s) ¹	Red (%)	Overhead (%)			Time (s)	Red (%)	Overhead (%)		
				SER	Area	Power			Delay	SER	Area
s298 (17, 20)	10	0.06	27.14	6.21	5.54	5.00	6.48	22.20	9.51	4.78	1.35
s344 (24, 26)	13	2.2	8.52	3.48	3.72	3.17	143.85	13.64	6.94	8.74	10.07
s386 (13, 13)	10	3.56	26.03	4.55	2.75	1.92	58.78	36.56	7.10	0.33	0.00
s444 (24, 27)	21	2.23	24.70	9.15	7.96	12.28	204.08	31.59	9.00	10.39	0.00
s641 (54, 43)	8	1.29	8.19	1.57	1.14	0.00	2233.14	11.75	5.54	5.25	5.27
s713 (54, 42)	16	30.12	13.40	2.30	1.71	0.47	2397.78	19.77	4.35	5.29	2.11
s953 (45, 52)	45	36.66	13.10	4.93	4.40	13.04	2695.38	48.85	12.64	6.95	9.92
s1238 (32, 32)	19	101.35	8.49	2.90	1.37	1.01	370.12	29.88	4.98	1.21	2.21
s1423 (91, 79)	71	739.83	19.94	5.15	4.06	1.96	974.86	21.79	10.92	8.58	5.34
s1488 (14, 25)	27	14.13	33.10	3.47	1.85	1.83	1741.31	39.25	5.54	1.29	6.51
s1494 (14, 25)	27	14.08	33.17	3.79	2.50	1.79	1733.47	39.88	6.31	0.01	2.06
Avg.	24.27	85.96	19.62	4.32	3.36	3.86	1141.75	28.65	7.53	4.80	4.08

¹ Time doesn't include the implications learning process.**Table 7**
Soft error rate reduction for large ISCAS benchmark circuits.

Circuit	Implications Applied until <i>Gain</i> < 0.02					
	#FRW	Time (s)	Red (%)	Overhead (%)		
				SER	Area	Power
s5378 (214, 213)	120	1956.78	28.36	15.51	11.20	8.39
s9234 (247, 250)	145	2781.23	40.12	22.66	18.10	11.37
s13207 (700, 790)	245	3169.51	39.36	25.32	14.56	12.30
s15850 (611, 684)	175	3546.23	28.77	21.67	17.89	9.95
s35932 (1763, 2048)	312	5978.87	30.33	19.87	8.85	10.51
Avg.		3486.52	33.39	21.01	14.12	10.50

inaccuracies when the implication path has gates with reconvergent fanouts. Our approach attempts to improve the accuracy by identifying the impacted gates along an implication path and taking fanouts of gates into consideration. Furthermore, the approach in [20] limits the addition of FRWs to two wires for any target gate and only uses implications that do not result in adding extra inverters. While this could reduce the area and delay overhead, it could have a limitation on enhancing circuit soft error tolerance.

Finally, to illustrate the scalability of our proposed implication-based learning technique, we have run experiments on some of the largest ISCAS 89 benchmark circuits as shown in Table 7. FRWs have been added based on learned implications until *Gain* < 0.02. As can be seen from the results, a 33.39% average reduction of SER is achieved with an average area overhead of 21.01%. The major bulk of execution time is consumed by the implication learning process.

6. Conclusion

In this work, we have proposed an integrated fault tolerance technique based on the combined application of an implication-based fault tolerance technique and selective transistor redundancy technique. We have proposed an enhanced implication-based fault tolerance technique. In order to reduce implication learning time, implications are identified between a set of candidate source and target gates. Then, for each implication the gain in reduction of gate fault detection profitabilities is estimated. The implication with the highest gain is selected and its corresponding functionally redundant wire is added. The process is repeated until the gain is less than a given threshold. In comparison to existing implication-based fault tolerance techniques, the proposed fault tolerant technique achieves higher soft error rate reduction for the same number of functionally redundant wires added

based on implications. The integrated application of the proposed implication based fault tolerance technique and the selective transistor redundancy technique achieves significantly lower area overhead in comparison to applying the selective transistor redundancy technique alone with the same achieved reliabilities.

Acknowledgement

The authors would like to acknowledge the support provided by the Deanship of Scientific Research at King Fahd University of Petroleum & Minerals (KFUPM) under Research Grant IN131014.

References

- [1] J.R. Heath, P.J. Kuekes, G.S. Snider, R.S. Williams, A defect-tolerant computer architecture: opportunities for nanotechnology, *Science* 280 (5370) (1998) 1716–1721.
- [2] N. Cohen, T. Sriram, N. Leland, D. Moyer, S. Butler, R. Flatley, Soft error considerations for deep-submicron cmos circuit applications, in: *Proceeding of the International Electron Devices Meeting, IEDM '99, Dec 1999*, pp. 315–318.
- [3] A. Namazi, M. Nourani, Reliability analysis and distributed voting for nmr nanoscale systems, in: *Proceedings of the 2nd International Design and Test Workshop, IDT'2007, Dec 2007*, pp. 130–135.
- [4] K. Mohanram, N. Touba, Partial error masking to reduce soft error failure rate in logic circuits, in: *Proceedings of the 18th IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems*, Nov 2003, pp. 433–440.
- [5] A.H. El-Maleh, F.C. Oughali, A generalized modular redundancy scheme for enhancing fault tolerance of combinational circuits, *Microelectron. Reliab.* 54 (1) (2014) 316–326.
- [6] A.H. El-Maleh, A.S. Al-Qahtani, A finite state machine based fault tolerance technique for sequential circuits, *Microelectron. Reliab.* 54 (3) (2014) 654–661.
- [7] Y. Dotan, N. Levison, D. Lilja, Fault tolerance for nanotechnology devices at the bit and module levels with history index of correct computation, *IET Comput. Digit. Tech.* 5 (4) (2011) 221–230.
- [8] J. Teifel, Self-voting dual-modular-redundancy circuits for single-event-transient mitigation, *IEEE Trans. Nucl. Sci.* 55 (6) (2008) 3435–3439.
- [9] M. Nicolaidis, Time redundancy based soft-error tolerance to rescue nanometer technologies, in: *Proceedings of the 17th IEEE VLSI Test Symposium, VTS'1999*, 1999, pp. 86–94.
- [10] S. N. Pagliarini, L. A. B. Naviner, J.-F. Naviner, Selective hardening methodology for combinational logic, in: *Proceedings of the 13th Latin American Test Workshop (LATW)*, 2012, pp. 1–6.
- [11] C. Zoellin, H. Wunderlich, I. Polian, B. Becker, Selective hardening in early design steps, in: *Proceedings of the 13th European Test Symposium*, 2008, pp. 185–190.
- [12] Q. Zhou, K. Mohanram, Gate sizing to radiation harden combinational logic, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 25 (1) (2006) 155–166.
- [13] C. Lazzari, G. Wirth, F. Kastensmidt, L. Anghel, R. Reis, Asymmetric transistor sizing targeting radiation-hardened circuits, *Electr. Eng.* 94 (1) (2012) 11–18.
- [14] A.T. Sheikh, A.H. El-Maleh, M.E.S. Elrabaa, S.M. Sait, A fault tolerance technique for combinational circuits based on selective-transistor redundancy, *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* 25 (1) (2017) 224–237.
- [15] K. Nepal, N. Alves, J. Dworak, R. I. Bahar, Using implications for online error detection, in: *IEEE International Test Conference*, 2008, ITC 2008, pp. 1–10.
- [16] N. Alves, A. Buben, K. Nepal, J. Dworak, R.I. Bahar, A cost effective approach for online error detection using invariant relationships, *IEEE Trans. Comput.-Aided*

- Des. Integr. Circuits Syst. 29 (5) (2010) 788–801.
- [17] N. Alves, Y. Shi, J. Dworak, R. I. Bahar, K. Nepal, Enhancing online error detection through area-efficient multi-site implications, in: Proceedings of the IEEE 29th VLSI Test Symposium (VTS), 2011, pp. 241–246.
- [18] R. I. Bahar, E. T. Lampe, E. Macii, Power optimization of technology-dependent circuits based on symbolic computation of logic implications, ACM Trans. Des. Autom. Electron. Syst., vol. 5, no. 3, pp. 267–293, Jul. 2000. [Online]. Available: (<http://doi.acm.org/10.1145/348019.348028>).
- [19] S. Almukhaizim, Y. Makris, Soft error mitigation through selective addition of functionally redundant wires, IEEE Trans. Reliab. 57 (1) (2008) 23–31.
- [20] B. Zhou, S. Thambipillai, W. Zhang, Soft error mitigation through selection of noninvert implication paths, in NASA/ESA Conference on Adaptive Hardware and Systems (AHS), 2014, pp. 77–82.
- [21] X. Han, M. Donato, R. I. Bahar, A. Zaslavsky, W. Patterson, Design of error-resilient logic gates with reinforcement using implications, in: Proceedings of the 26th Edition on Great Lakes Symposium on VLSI, ser. GLSVLSI '16. New York, NY, USA: ACM, 2016, pp. 191–196. [Online]. Available: (<http://doi.acm.org/10.1145/2902961.2902983>).
- [22] M. Donato, F. Cremona, W. Jin, R. I. Bahar, W. Patterson, A. Zaslavsky, J. Mundy, A noise-immune sub-threshold circuit design based on selective use of schmitt-trigger logic, in Proceedings of the Great Lakes Symposium on VLSI, ser. GLSVLSI '12. New York, NY, USA: ACM, 2012, pp. 39–44. [Online]. Available: (<http://doi.acm.org/10.1145/2206781.2206792>).
- [23] H.K. Lee, D.-S. Ha, Hope An efficient parallel fault simulator for synchronous sequential circuits, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 15 (9) (1996) 1048–1058.
- [24] H. Fujiwara, T. Shimono, On the acceleration of test generation algorithms, IEEE Trans. Comput. C-32 (12) (1983) 1137–1144.
- [25] M.H. Schulz, E. Trischler, T.M. Sarfert, Socrates: a highly efficient automatic test pattern generation system, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 7 (1) (1988) 126–137.
- [26] M.H. Schulz, E. Auth, Improved deterministic test pattern generation with applications to redundancy identification, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 8 (7) (1989) pp. 811–816.
- [27] W. Kunz, D.K. Pradhan, Recursive learning: a new implication technique for efficient solutions to cad problems-test, verification, and optimization, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 13 (9) (1994) 1143–1158.
- [28] Predictive technology model for spice, (<http://ptm.asu.edu/>).
- [29] Lgsynth'91 benchmark circuits, (<http://ddd.fit.cvut.cz/prj/benchmarks/>).
- [30] R.K. Brayton, A.L. Sangiovanni-Vincentelli, C.T. McMullen, G.D. Hachtel, Logic Minimization Algorithms for VLSI Synthesis, Kluwer Academic Publishers, Norwell, MA, USA, 1984.
- [31] E. Sentovich, K. Singh, L. Lavagno, C. Moon, R. Murgai, A. Saldanha, H. Savoj, P. Stephan, R. K. Brayton, A. L. Sangiovanni-Vincentelli, Sis: A system for sequential circuit synthesis, EECS Department, University of California, Berkeley, Tech. Rep. UCB/ERL M92/41, 1992.